Week 1 Module – C & C++ Programming

Topics to be Covered

Students are required to Submit handwritten notes and practice for the following topics in **C and C++ Programming Language**:

1. **Syntax**
2. **Conditions and Loops**
3. **Programming Problems**
    - Solve the given **Star Pattern Questions**

1. Square

```
*****
*****
*****
*****
*****
```

2. Hollow Square

```
*****
*   *
*   *
*   *
*****
```

3. Triangle

```
*
**
***
****
*****
```

4. Inverted Triangle

```
*****
****
***
**
*
```

5. Pyramid

```
    *
   ***
  *****
 *******
*********
```

6. Inverted Pyramid

```
*********
 *******
  *****
   ***
    *
```

7. Half Diamond

```
*
**
***
****
*****
****
***
**
*
```

8. Diamond

```
    *
   ***
  *****
 *******
*********
 *******
  *****
   ***
    *
```

C and C++ Programming

C Programming

(1) C Syntax and Basic structure

C is a procedural and structured programming language. Every standard C program has a precise, structured layout.

Header files (#include <stdio.h>)

This pre-processor directive includes the standard input/output library. It allows the program to use the core functions, such as printf() for output and scanf() for input.

Main function (int main())

This is the most mandatory entry point of any C program. Execution starts at the opening brace { of main(). Execution ends at the terminating semicolon of return 0;.

Output function (printf)

Unlike C++, C uses printf() function to display text on the screen. Double quotes let you pass format specifiers or escape sequences (\n (newline).

Semicolon (;): Every statement in C must end with a semicolon. It is a statement terminator for the compiler.

(2) Conditions and loops

Conditional statements (if-else): Used to make decisions. The if block runs when the condition is true. If it is false, the program skips it or executes the optional else block.

Loops (for loop): To repeat a block of code a certain number of times. It is composed of initialization, condition checking and increment/decrement expressions.

Nested loops: for star patterns to print two dimensional and patterns. we need a loop in another loop.

Outer loop: Manage the rows (vertical progression)

Inner loop: Handles the columns (prints spaces or stars horizontally for the current row).

(3) Programming problems

Solve the given star pattern Questions

(1) square

o/p:—

```
#include <stdio.h>
int main() {
    int n = 5;  // total rows and columns
    // outer loop for rows
    for (int i = 1; i <= n; i++) {
        // inner loop for columns
        for (int j = 1; j <= n; j++) {
            printf("*");
        }
        printf("\n");  // move to the next line
    }
    return 0;
}
```

main.c

Share Run

```c
#include <stdio.h>

int main() {
    int n = 5;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            printf("* ");
        }
        printf("\n");
    }

    return 0;
}
```

Output

Clear

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *


=== Code Execution Successful ===
```

② Hollow square pattern

```
#include <stdio.h>
int main () {
 int n = 5;
for (int i = 1; i <= n; i++) {
  for (int j = 1; j <= n; j++) {
    if (i == 1 || i == n || j == 1 || j == n) {
      printf ("*");
    }
    else {
      printf (" ");    // empty space
    }
  }
  printf ("\n");     // next line.
}
return 0;
}
```

o/p :-

```
* * * * *
*       *
*       *
*       *
* * * * *
```

main.c

Share Run

```c
#include <stdio.h>

int main() {
    int n = 5;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {

            // Print star only at the boundaries
            if (i == 1 || i == n || j == 1 || j == n) {
                printf("*");
            } else {
                printf(" "); // Print empty space inside
            }
        }

        printf("\n");
    }

    return 0;
}
```

Output

Clear

```
*****
*   *
*   *
*   *
*****


=== Code Execution Successful ===
```

(3) Triangle

```
#include <stdio.h>
int main () {
  int n = 5;
  for (int i = 1; i<=n; i++) {
     // inner loop : print stars in increasing orde
     for (int j = 1; j<=i; j++) {
        printf (" * ");
     }
     // move to the next line
     printf ("\n");
  }
  return 0;
}
```

O/p:-

```
*
* *
* * *
* * * *
* * * * *
```

main.c

Share Run

```c
#include <stdio.h>

int main() {
    int n = 5;

    for (int i = 1; i <= n; i++) {

        // Inner loop runs up to the current row number 'i'
        for (int j = 1; j <= i; j++) {
            printf("*");
        }

        printf("\n");
    }

    return 0;
}
```

Output Clear

```
*
**
***
****
*****


=== Code Execution Successful ===
```

(4) Inverted Triangle

```
#include <stdio.h>
int main () {
  int n = 5;
  // outer loop counts down from 'n' to 1
  for (int i = n; i >= 1; i--) {
      // inner loop: print 'i' stars
      for (int j = 1; j <= i, j++) {
          printf("*");
      }
      printf("\n");
  }
  return 0;
}
```

o/p:-

```
* * * * *
 * * * *
  * * *
   * *
    *
```

main.c

Share Run

```c
#include <stdio.h>

int main() {
    int n = 5;

    // Outer loop counts down from 'n' to 1
    for (int i = n; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            printf("*");
        }

        printf("\n");
    }

    return 0;
}
```

Output Clear

```
*****
****
***
**
*


=== Code Execution Successful ===
```

(5) Pyramid

```
#include <stdio.h>
int main () {
  int  n = 5;
  for (int  i = 1;  i <= n;  i++) {
     // print leading spaces
     for (int  j = 1;  j <= n-i; j++) {
          printf (" ");
     }
     // print stars based on odd numbers formula
     for (int  k = 1;  k <= (2*i-1); k++) {
          printf("*");
     }
     printf("\n");
  }
  return 0;
}
```

O/P:-

```
        *
      * * *
    * * * * *
  * * * * * * *
* * * * * * * *
```

main.c

Share Run

```c

int main() {
    int n = 5;

    for (int i = 1; i <= n; i++) {

        // Print leading spaces
        for (int j = 1; j <= n - i; j++) {
            printf(" ");
        }

        // Print stars
        for (int k = 1; k <= (2 * i - 1); k++) {
            printf("*");
        }

        printf("\n");
    }

    return 0;
}
```

Output

Clear

```
    *
   ***
  *****
 *******
*********


=== Code Execution Successful ===
```

(6) Inverted pyramid

```
#include <stdio.h>
int main () {
   int n = 5;
   // outer loop handles rows in reverse order
   for (int i = n; i>=1; i--) {
       // print leading spaces
       for (int j = 1; j<=n-i; j++) {
           printf(" ");
       }
       // print stars
       for (int k = 1; k<= (2*i-1); k++) {
           printf("*");
       }
       printf("\n");
   }
   return 0;
}
```

O/p:-

```
* * * * *
 * * * *
  * * *
   * *
    *
```

main.c

Share Run

```c
int main() {
    int n = 5;

    // Outer loop
    for (int i = n; i >= 1; i--) {

        // Print spaces
        for (int j = 1; j <= n - i; j++) {
            printf(" ");
        }

        // Print stars
        for (int k = 1; k <= (2 * i - 1); k++) {
            printf("*");
        }

        printf("\n");
    }

    return 0;
}
```

Output

Clear

```
*********
 *******
  *****
   ***
    *


=== Code Execution Successful ===
```

(7) Half diamond

```
#include <stdio.h>
int main () {
   int n = 5;
    // upper half : Creating triangle
    for (int i = 1; i<=n; i++) {
        for (int j = 1; j<= i; j++) {
            printf("*");
        }
        printf ("\n");
    }
   // lower half : Shrinking inverted triangle
   for (int i = n-1; i >= 1; i--) {
       for (int j = 1; j<= i; j++) {
           printf("*");
     }
      printf ("\n");
  }
  return 0;
}
```

O/p:-

```
*
* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*
```

main.c

Share Run

```c
int main() {
    int n = 5;

    // Upper half
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }

    // Lower half
    for (int i = n - 1; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }

    return 0;
}
```

Output

Clear

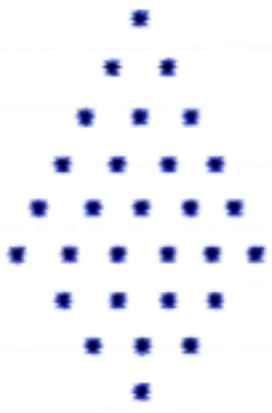
```
*
**
***
****
*****
****
***
**
*


=== Code Execution Successful ===
```

(8) Diamond

```
#include <stdio.h>
int main() {
    int n = 5;
    // upper half : Regular pyramid
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n-i; j++) {
            printf(" ");
        }
        for (int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }
    // lower half : Inverted pyramid
    for (int i = n-1; i >= 1; i--) {
        for (int j = 1; j <= n-i; j++) {
            printf(" ");
        }
        for (int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```

O/P:-



main.c

Share Run

```c
#include <stdio.h>

int main() {
    int n = 5;

    // Upper half
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n - i; j++)
            printf(" ");

        for (int k = 1; k <= (2 * i - 1); k++)
            printf("*");

        printf("\n");
    }

    // Lower half
    for (int i = n - 1; i >= 1; i--) {
        for (int j = 1; j <= n - i; j++)
            printf(" ");

```

```
        printf(" ");

    for (int k = 1; k <= (2 * i - 1); k++)
        printf("*");

    printf("\n");
}

// Lower half
for (int i = n - 1; i >= 1; i--) {
    for (int j = 1; j <= n - i; j++)
        printf(" ");

    for (int k = 1; k <= (2 * i - 1); k++)
        printf("*");

    printf("\n");
}

return 0;
}
```

Output

Clear

```
    *
   ***
  *****
 *******
*********
 *******
  *****
   ***
    *


=== Code Execution Successful ===
```